

오픈소스 반도체 설계 및 제작 종합설계 2주차 학습노트

이수빈

과제1. 텍스트 파일 비교

두 버전의 차이는 다음 명령으로 비교했다.

```
$ diff -y -W 128 ../Q.09c_Enhance_Keyboard_Stim/sc_invaders_TB.cpp
./sc_invaders_TB.cpp | less
$ diff -y -W 128 ../Q.09d_Enhance_GLCD_TLM/sc_glcd128x64_TLM.cpp
./sc_glcd128x64_TLM.cpp | less
```

주석이 많이 바뀌어 코드가 실제로 달라진 곳이 잘 보이지 않아서, 한 줄 주석을 지우고 공백 차이를 무시하는 명령을 추가로 돌려 로직만 비교했다.

sc_invaders_TB.cpp는 Q.09c와 Q.09e 사이에서 코드가 달라진 곳이 리셋 해제 후 출력하는 로그 문구 한 줄뿐이었다. 클럭 생성, 리셋 시퀀스, 60Hz 폴링, 키 매핑은 그대로였고, 나머지는 주석 차이였다. sc_glcd128x64_TLM.cpp도 Q.09d와 Q.09e 사이에서 에러 메시지 두 개와 창 종료 로그 한 줄의 문구만 바뀌었고 로직은 같았다.

기능이 실제로 바뀐 것은 그 앞 단계였다. sc_invaders_TB.cpp는 Q.09b에서 Q.09c로 넘어가며 키 상태를 읽기 전에 SDL_PumpEvents()로 이벤트를 갱신하게 되었고, 키보드 상태 포인터의 nullptr 검사와 리셋 해제 로그가 추가되었다. sc_glcd128x64_TLM.cpp는 Q.09b에서 Q.09d로 넘어가며 SDL_INIT_EVENTS 초기화, 창 생성 실패 검사, V-Sync 렌더러와 소프트웨어 렌더러 폴백이 추가되었고, 창을 닫을 때 sc_stop()만 호출하던 것이 Verilated::gotFinish(true)를 함께 호출하도록 바뀌었으며 이를 위해 <verilated.h>가 추가되었다.

정리하면 기능이 바뀐 것은 Q.09c(키보드 입력)와 Q.09d(GLCD 안정성과 종료 연동)이고, Q.09e는 그 결과를 제출 가능한 상태로 정리한 버전이었다.

과제2. Makefile 수정

make -f Makefile.mk를 돌렸을 때 어떤 순서로 막혔었는지를 기준으로 아래를 기록해보았다.

1) 파일의 누락

가장 먼저 make -f Makefile.mk를 실행하자 Verilator 번역 단계에서 곧바로 멈췄다. 첫 오류 줄은 Cannot find file containing module: 'space_invaders_engine_glcd.v'였고, 아래에 찾아본 위치가 나열되어 있어서 이 파일이 폴더에 없다는 것을 바로 알 수 있었다. 이에 따라 이전 과제인 Q.09b에서 .v 파일을 복사해 왔다.

2) 타겟 이름 불일치

파일을 채워 넣고 다시 빌드하자, 이번에는 번역이 진행되면서 WIDTHTRUNC, BLKSEQ 같은 경고 메시지가 화면을 가득 채웠다. 처음에는 쏟아지는 빌드 로그가 너무 길어서 윗부분에 있는 핵심 오류만 빠르게 파악하려고 head -30 명령어를 사용해 로그를 잘라 보았다. 하지만 Makefile에 -Wno-fatal 옵션이 있어서 수많은 경고에도 불구하고 번역 과정이 멈추지 않았고, 정작 에러 메시지는 잘려나간 로그 뒷부분에 있어서 처음에는 마치 오류가 없는 것처럼 보였다. 다시 확인해보니, 15번째 줄 즈음에 No rule to make target 'Vspace_invaders_engine.mk'라는 오류가 숨어 있었다. Verilator는 파일 이름이 아니라 최상위 모듈 이름으로 makefile을 만들기 때문에 실제 생성된 이름은 Vspace_invaders_engine_glcd.mk였다. Makefile.mk의 타겟과 바이너리 이름을 알맞게 수정했다.

3) 컴파일 단계와 나머지 파일 누락 문제

이름을 고치자 처음으로 2단계 컴파일 과정에 진입했는데, 이번에는 sc_main.cpp와 헤더 두 개(sc_glcd128x64_TLM.h, sc_invaders_TB.h)가 없다는 오류 세 개가 한꺼번에 튀어 나왔다. 이 역시 1번과 같이 세 파일을 한 번에 복사해 왔다.

4) 포트 타입 불일치

모든 파일이 채워지고 다시 컴파일이 시작되자, 이번엔 또 오류 메시지가 터미널 화면을 꽉 채울 만큼 길게 쏟아졌다. 로그가 너무 길어서 뭔가 크게 잘못된 줄 알았지만, 오류는 맨 윗줄 하나뿐이었다. 나머지는 컴파일러가 후보를 하나씩 시도해 본 부연 설명에 불과해서 허무한 기분도 들었다. 진짜 원인은 sc_main.cpp:43:20: error: no match for call to '(sc_core::sc_out<unsigned int>) (sc_core::sc_signal<sc_dt::sc_uint<7>>&)'였다. Verilator는 별도 옵션이 없으면 2비트 이상 포트를 일반 정수형으로 생성하는데, sc_main.cpp는 그 포트를 SystemC 전용 타입인 sc_uint 신호에 연결하려다 타입이 맞지 않아 발생한 문제였다. 실패한 것은 sc_main.o 하나뿐이었기에, Verilator 플러그에

—pins-sc-uint를 추가하고 다시 빌드했다. 이 한 가지만 고쳤는데 그 길었던 오류가 전부 사라졌다.

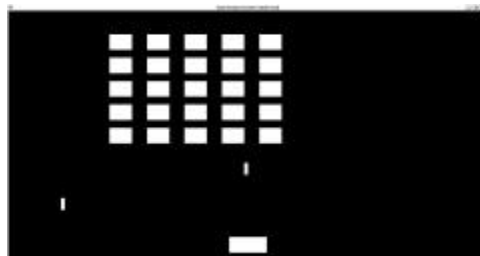
5) 실행 결과

완성된 시뮬레이터(./obj_dir/Vspace_invaders_engine_glcd)를 실행하니 128x64창이 뜨고 화면이 정상적으로 그려졌다. 방향키와 스페이스바에 게임이 잘 반응했고, 창을 닫자 SDL_QUIT 처리가 verilated::gotFinish(true)와 sc_stop()으로 정상적으로 이어지며 프로세스가 깔끔하게 종료되었다.

```
soobin@localhost:~/ETR1858_DesignKit/Projects/MyChip_Games/space_invader_by_Gemini/Q.09e_SystemC_Env_Quiz$ ./obj_dir/Vspace_invaders_engine_glcd

SystemC 3.0.2-Accellera --- Aug 3 2026 02:56:35
Copyright (c) 1996-2025 by all Contributors,
ALL RIGHTS RESERVED
[SystemC TB] 하드웨어 전원 리셋 해제 완료, 에뮬레이터 키보드 제어권을 활성화합니다.
[GLCD] 사용자가 시뮬레이터 창을 닫아 프로세스를 조기 종료합니다.
Info: /OSCI/SystemC: Simulation stopped by user.
```

▲ 완료된 로그



▲ 실행모습

과제를 수행하며: Space Invader 라는 게임의 '느낌'을 가졌다는 것으로 개발 (설계) 했다고 할 수 있을까?

설계는 내가 원하는 동작, 즉 아이디어를 기술하는 것에서 시작하고, 우리의 목표는 그 아이디어를 칩으로 만드는 것이라고 배웠다. 이 관점에서 Space Invaders의 '느낌'을 내는 것은 설계의 첫 단계, 곧 아이디어가 원하는 대로 동작하는지 눈으로 확인하는 일이다. 그 느낌을 내는 것은 Gemini의 도움으로 어렵지 않았다. 실제로 창이 뜨고 방향키와 스페이스바에 게임이 반응했다. 다만 나는 그 느낌만으로는 설계했다고 하기 어렵다고 생각한다. 여기서 '느낌'은 시뮬레이션에서 게임처럼 보이고 반응하는 것이고, '설계'는 그것이 왜 그런 구조로 돌아가는지 알고 조건이 바뀌어도 맞는지 판단하는 것이라고 생각한다. 칩 면적과 동작 속도, 타이밍에 여유가 있는 환경에서는 느낌만으로도 게임을 만들 수 있지만, 제약이 뻑뻑해질수

록 필요한 것은 느낌이 아니라 그 뒤에 있는 구조와 이유를 아는 것이다.

이번 과제에서 내가 코드의 로직을 수정한 일은 없었다. 그런데도 코드만으로는 바로 돌아가지 않았다. 내가 한 일은 코드가 돌아가는 환경과 코드를 연결하는 일이었다. 빠진 파일을 채우고, 타깃 이름을 Verilator가 만든 이름에 맞추고, SystemC 경로를 실제 설치 위치로 고쳤다. 특히 Verilator가 만든 엔진의 포트와 sc_main.cpp를 연결하자 포트 타입이 맞지 않는다는 오류가 화면을 가득 채울 만큼 길게 나왔고, 빌드 옵션 하나(--pins-sc-uint)를 추가해서야 해결되었다. 각각은 정상으로 보이는 조각도 이어 붙이는 자리에서 어긋날 수 있다는 것을 확인했다. 게다가 이 결과는 오직 내 컴퓨터의 시뮬레이션 환경에서 돌아간 것일 뿐이다. 이 프로젝트에서 실제로 칩이 되는 것은 Verilog로 쓴 게임 엔진이고, SDL 창과 키보드 입력은 시뮬레이션을 위한 테스트벤치다. 그래서 시뮬레이션에서 게임처럼 보였다는 사실은, 그 엔진이 칩으로 만들어졌을 때 면적과 타이밍, 전력 제약 안에서 동작하는지에 대해서는 아직 아무것도 말해 주지 않는다.

실제로 이번 빌드에서 Verilator는 MULTIDRIVENPROC, BLKSEQ 같은 경고를 길게 냈다. MULTIDRIVENPROC는 하나의 변수를 둘 이상의 always 블록이 쓰고 있다는 경고다. 레지스터 하나는 보통 하나의 always 블록이 갱신하는데, 두 블록이 같은 변수를 쓰면 시뮬레이션에서는 결과가 실행 순서에 좌우되고, 합성에서는 한 신호를 두 곳에서 구동하게 되어 문제가 될 수 있다. BLKSEQ는 클럭에 맞춰 동작하는 순차 로직 안에서 블로킹 할당(=)을 썼다는 경고다. 순차 로직의 레지스터는 보통 논블로킹 할당(<=)으로 갱신하는데, =는 값이 그 자리에서 바로 바뀌기 때문에 같은 클럭 엣지에서 그 값을 읽는 다른 블록이 예전 값을 볼지 새 값을 볼지가 실행 순서에 따라 달라질 수 있다.

이 경고들을 무시한 채로 칩을 제작해보지는 않았기에, 이것이 실제로 문제가 되는지는 확인하지 못했다. 하지만 시뮬레이션에서는 통과된다 하더라도 하드웨어로 합성할 때는 다르게 취급될 수 있는 종류의 경고라는 것은 알고 있다. 코드가 돌아가는 것과 칩으로 만들 수 있는 것은 다른 일이다.

이번 프로젝트는 그런 제약이 넉넉한 환경이었다. 내가 사용한 킷은 0.5um CMOS 공정용이고, 테스트벤치의 클럭도 반주기 10ns, 즉 50MHz로 느긋하다. 화면도 128x64 픽셀로 작고, 시뮬레이션에서 돌아가면 충분한 과제였다. 이렇게 칩 면적에 여유가 있고, 동작 속도가 느리고, 타이밍을 깊게 따지지 않아도 되는 조건에서는 AI가 준 코드를 돌려 보고 게임의 느낌이 나는지만 확인해도 유쾌하게 코딩할 수 있다고 생각한다. 하지만 지금의 산업은 그 반대 방향으로 움직이고 있다. 공정은 미세해지고, 클럭은 빨라지고, 같은 면적에 더 많은 기능을 넣어야 하며, 전력까지 함께 맞춰야 한다. 이런 환경에서는 느낌을 확인하는 것만으로는 충분하지 않다.

앞에서 본 MULTIDRIVENPROC, BLKSEQ 경고도 시뮬레이션에서는 게임이 잘 돌아갔기 때문에 그냥 지나칠 수 있었다. 여유가 큰 조건에서는 이런 것들이 드러나지 않지만, 여유가 없

는 조건에서는 실행 순서에 기대는 코드나 한 신호를 두 곳에서 쓰는 구조가 실제 문제가 될 수 있다고 알고 있다. 그래서 나는 느낌만으로 하는 설계에는 산업 현장에서 굉장히 큰 맹점이 있다고 생각한다. 느낌은 여유 있는 조건에서나 통하는 감각이고, 조건이 뻑뻑해질수록 그 뒤에 있는 구조와 이유를 알고 있어야 하기 때문이다.

여유 있는 조건이 문제를 가려 주는 일은 칩의 제약에만 있지 않았다. 내가 직접 확인한 가장 작은 예는 SystemC 경로였다. 다른 학우의 과제에는 SystemC 경로를 실제 설치 위치로 고친 내용이 기록되어 있었는데, 나는 그 경로를 고치기 전의 원본 상태에서도 빌드와 실행이 성공했다. 이유를 찾아보니 Verilator가 이미 SystemC 위치를 /opt/systemc로 알고 있어서 컴파일과 링크 때 그 경로를 자동으로 붙여 주었고, 실행할 때는 LD_LIBRARY_PATH가 설정되어 있어서 라이브러리를 찾을 수 있었다. 내 환경에서는 고치지 않아도 성공했지만, 그것은 Makefile이 옳아서가 아니라 내 컴퓨터의 설정이 대신 메워 주었기 때문이다. 그래서 이 수정은 내 환경에서는 필수가 아니었지만, Makefile이 실제 설치 위치를 가리키도록 고쳐 둔 것은 의미가 있었다고 생각한다.

내 환경에서만 돌아가는 것과 다른 환경, 그리고 다양한 제약이 넘쳐나는 실제 칩에서도 맞게 동작하는 것 사이의 차이를 스스로 확인하고 메우는 일이 엔지니어가 해야 할 일이라고 생각한다.